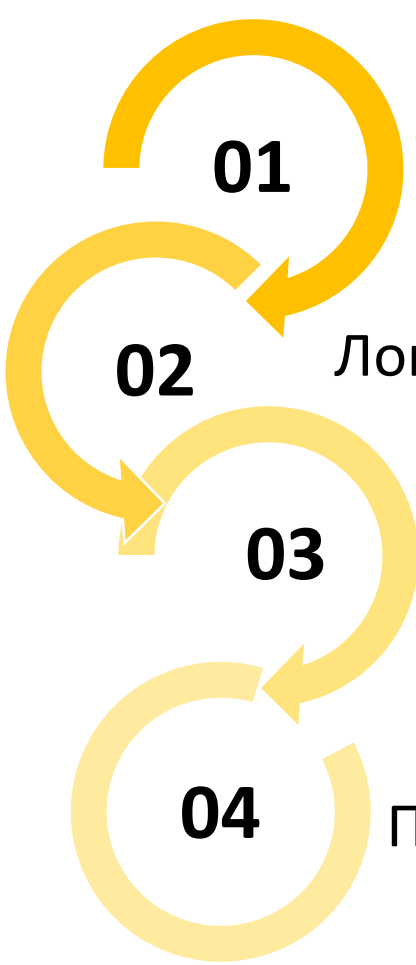

Яндекс

Яндекс Лицей

Условный оператор и не только —

как заставить код принимать решения

Что впереди

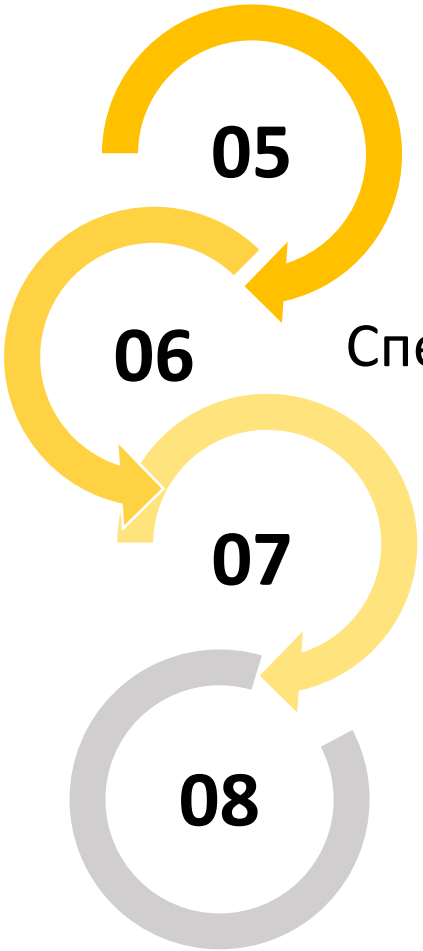


01 Условный оператор
Выбор как в RPG

02 Логический тип данных
Булева логика

03 Логические операторы
И сложные условия

04 Подстрока
Есть или нет?



05 Пустая инструкция
pass

06 Спецсимволы
Наследие печатных машинок

07 match/case
Выбираем со стилем

08 **Итоги**



Условный оператор

Выбор как в RPG

Условный оператор



Представьте, что вам нужно написать развилку в квесте:
— Если пойдёте налево — найдёте сундук с золотом.
— Иначе — нарвётесь на тролля.

В Python для этого есть специальный оператор условия if-else.

```
weather = input()
if weather == "дождь":
    print("Бери зонт и резиновых уточек! 🐥 ")
else:
    print("Солнцезащитные очки — твой выбор!")
print("Хорошего дня!")
```

Важно:

- строчка с оператором обязательно заканчивается двоеточием,
- с новой строки идёт код с отступом в четыре пробела.
- после блочного оператора должна быть минимум одна строка вложенного кода, но может быть сколько угодно, в том числе другие блочные операторы.

Важно:

Если if может быть без else-части, то вот else не может существовать без своего if.

Еда может существовать без котика, а вот котик без еды — нет!



Логические операторы

Чёрно-белый мир булевой логики

Логические операторы



Операторы отношений:

`==` — равно (потому что один знак `=` — оператор присваивания)
`!=` — не равно
`>` — больше
`>=` — больше или равно

(пишется как читается)

`<` — меньше
`<=` — меньше или равно

PEP8
Вокруг операторов отношений обязательно нужно ставить по пробелу с каждой стороны.

Тип данных bool:
у переменных этого типа только два значения

- **True** — «Да, я съел всю пиццу».
- **False** — «Нет, это был кот!».

```
print(2 == 2)  # True
print(3 < 1)  # False
print(5 <= 13)  # True
print("Привет" == 'привет’)
# False — регистр букв имеет значение!
```

Python поддерживает **двойные операции отношения**:

```
x_coord = int(input())
if 0 <= x_coord < 100:
    print('Координата в допустимом диапазоне')
```

Логический тип данных

Значения типа bool можно задавать и напрямую из кода:

```
is_pizza_gone = True  # Истинно всё съели
pizza_gone = 'True'  # Строка "True"
print(is_pizza_gone == pizza_gone)
# Конечно же, значения не равны — False
```

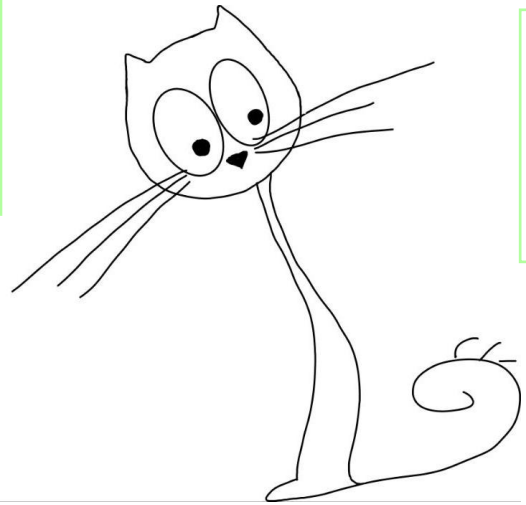
Булевы переменные можно использовать в if-else:

```
is_cat_awake = True  # Кот не спит → можно гладить
if is_cat_awake:
    print("😺 Мур-мур!")
else:
    print("💤 Кот в отключке.")
```

В Python почти всё можно превратить в bool:

- **False:** 0, пустая строка (правильнее сказать — любая пустая коллекция);
- **True:** всё остальное (даже отрицательные числа! и даже 0.000000001!).

Это можно сделать явно с помощью функции bool():



```
print(bool('😺'))  # True
print(bool(''))  # False
print(bool(0))  # False
print(bool(-3.14))  # True
```

Логический тип данных

Похожее приведение происходит неявно в операторе if — значение оригинальной переменной не меняется, но воспринимается программой как True/False:

```
print("Как зовут вашего котика?")
pet_name = input()
print("А сколько ему лет?")
pet_age = int(input())

if pet_name:  # Пользователь ввёл любое имя
    print(f"Какое красивое имя — {pet_name}")
else:  # Пользователь просто нажал Enter, и в pet_name — пустая строка
    print("Безымянный котик — тоже котик")

if pet_age:  # Любое ненулевое число
    print("Уже большой!")
else:  # Котику 0 лет
    print('Какой малыш!')
```



Мы добавили вам условие в условие



Внутри любой части оператора if-else может быть другой оператор if-else, а внутри второго — третий и так далее.

```
age = 25
has_money = True

if age >= 18:
    if has_money:
        print("Можно отдохнуть.")
    else:
        print("Ищи работу! 💼 ")
else:
    print("Иди учи уроки!")
```

После if может идти сколько угодно операторов elif, условия будут проверяться сверху вниз, и проверка прекратится, как только одно из условий будет истинным.

Каждый вложенный блок внутри вложенного блока прибавляет ещё 4 пробела к отступу.
Можно использовать оператор elif.

```
age = int(input("Сколько лет тебе, путник?"))

if age >= 3000:
    print("О, великий древний! Расскажи про Python 0.1!")
elif age >= 100:
    print("Добро пожаловать в музей COBOL.")
else:
    print("Юнец, иди учить PEP8!")
```

Логическое трио: not, and, or

Следите за руками (и условиями)

Другим способом избежать вложенных условий являются логические операции, которые позволяют делать сложные условия. Вот они в порядке уменьшения приоритета:

- **not** — «не». Инвертирует истину в ложь и наоборот. Срабатывает всегда первым.

```
if not is_cat_hungry:
    print("Кот сыт. Все живы!")
```

- **and** — «и». Чтобы получить истину, надо, чтобы обе части были истинны, иначе — False.

```
if name == "Цезарь" and password == "Ave123":
    print("Добро пожаловать в императорский чат.")
```

- **or** — «или». True получится, даже если хотя бы одно из условий будет истинным.

```
if is_weekend or is_holiday:
    print("Ура, можно валяться в гамаке!")
```

Логическое трио: not, and, or

Чтобы повысить приоритет операций, нужно использовать скобки, точно так же, как и в математике. Несколько примеров.

```
print("Идёт дождь?")
is_rain = input() == 'да'
print("А холодно?")
is_cold = input() == 'да'

if is_rain and not is_cold:  # Дождь и не холодно
    print("Зонт и шлёпанцы — ваш выбор! (Дождь, но тепло!)")
elif is_rain and is_cold:  # И дождь, и холодно
    print("Куртка, зонт и горячий чай в термосе!")
else:
    print("Панама и уверенность в себе!")
```



У оператора присваивания самый низкий приоритет, поэтому сначала выполнится input(), потом пользовательский ввод сравнится со строкой «да» и только потом результат сравнения True|False запишется в переменную.

Логическое трио: not, and, or

Иногда нужно посмотреть на условие немного под другим углом, и код сразу станет проще и понятнее. Например, нам *надо ввести чётное число, которое больше 3000 и при этом ещё делится на пять или три.*

Можно написать что-то вот такое:

```
num = int(input())
if num % 2 == 0 and num > 3000 and (num % 3 == 0 or num % 5 == 0):
    print("Ура!")
    print(f"{num} — подходит")
elif num % 2 != 0:
    print("Нечётное!")
elif num <= 3000:
    print("Меньше или равно 3000")
elif not (num % 3 == 0 or num % 5 == 0):
    print("Не делится ни на три, ни на пять")
```

Но если немного подумать, то можно понять, что если все проверки в elif пройдены, то это число и есть искомое. Тогда можно переписать вот так:

```
num = int(input())
if num % 2 != 0:
    print("Нечётное!")
elif num <= 3000:
    print("Меньше или равно 3000")
elif not (num % 3 == 0 or num % 5 == 0):
    print("Не делится ни на три, ни на пять")
else:
    print("Ура, {num} — подходит")
```

Вот это понаставили условий



Для переноса длинной строки условий используются круглые скобки:

```
if (user_age > 12 and
      user_balance >= 1000 and
      not is_banned):
   print("Доступ разрешён!")
```

Кроме того, можно записать результат выполнения условия или его части в переменную ещё до оператора if для того, чтобы сократить запись и сделать код более читаемым. Сравните:

```
print('Введите имена двух братьев из мифов, кино или видеоигр:')
brother1 = input()
brother2 = input()

if (brother1 == 'Ромул' and brother2 == 'Рем' or
      brother1 == 'Кастор' and (brother2 == 'Поллукс' or
                                brother2 == 'Полидевк') or
      brother1 == 'Тор' and brother2 == 'Локи' or
      brother1 == 'Марио' and brother2 == 'Луиджи'):
   print('✅ Ага, знаток! Даже поп-культуру захватил!')
else:
   print('❌ Неа. Может, вы перепутали Кастора с Человеком-пауком?')
```

И вариант с записью частей условий в переменную:

```
print('Введите имена двух братьев из мифов, кино или видеоигр:')
brother1 = input()
brother2 = input()

is_roman = brother1 == 'Ромул' and brother2 == 'Рем'
is_greek = brother1 == 'Кастор' and (brother2 == 'Поллукс' or
                                     brother2 == 'Полидевк')
is_scandinavian = brother1 == 'Тор' and brother2 == 'Локи'
is_marioworld = brother1 == 'Марио' and brother2 == 'Луиджи'

if is_roman or is_greek or is_scandinavian or is_marioworld:
   print('✅ Ага, знаток! Даже поп-культуру захватил!')
else:
   print('❌ Неа. Может, вы перепутали Кастора с Человеком-пауком?')
```

Когда ничего не хочется прямо сейчас



Внутри блочного оператора должна быть как минимум одна строчка кода. Но бывает, что мы знаем условие, а вот обработку того, что будет происходить при его истинности, пока не придумали.

В этом случае можно воспользоваться специальной командой-заглушкой pass, которая ничего не делает, но при этом наш код не будет падать с синтаксической ошибкой.

```
if user_status == "admin":
    # TODO: Добавить права админа
    pass
else:
    print("Доступ запрещён! 🛑 ")
```

Иногда в качестве заглушки используется оператор, который называется ellipsis, а записывается как троеточие .... Это тоже валидно.

```
if conquer_the_wold:
    # Потом. Сегодня лень.
    ...
```

В некоторых командах разработки договариваются вкладывать в pass и ... немного разный смысл:

- pass — заглушка, но так и должно быть (такое бывает)
- ... — заглушка, но потом тут должен появиться осмысленный код

Немного о строках

и всём, что с ними связано

Подстрока есть? А если найду?



В Python можно легко проверить, является ли какая-нибудь строка подстрокой другой. Для этого используется оператор in.

```
text = input()
if "кот-учёный" in text:
    print("Найден кот-ботаник!")
```

Ещё пример:

```
password = input()

if len(password) >= 8 and (
        "!" in password or "#" in password):
    print("Надёжный пароль! 🔒 ")
else:
    print("Слабовато...")
```

Чтобы проверить невхождение подстроки в строку, пишут not in.

```
text = input()
if "дракон" not in text:
    print("Драконов нет. Только подземелья.")
```

Хотя синтаксически не будет ошибкой написать not ("дракон" in text), в приличном обществе так никто не делает. Как минимум потому, что по-русски мы говорим не "не подстрока в строке", а "подстрока не в строке".

Спецсимволы, или Наследие печатных машинок



Строки в Python (и других языках программирования) могут содержать управляющие последовательности, которые начинаются со знака \. Вот самые важные из них:

- \n — перенос строки;
- \r — возврат каретки
  (устаревшее, но до сих пор встречается);
- \t — табуляция.

```
print("Первая строка\nВторая строка\tТабуляция")
```

```
Первая строка
Вторая строка	Табуляция
```

В современных операционных системах \n объединяет в себе функцию переноса строки и возврата каретки, а раньше строчки в тексте заканчивались комбинацией \n\r, что имитировало работу на печатной машинке: \n переносил курсор на следующую строчку, а \r возвращал его на начало строки.

Экранирование:

```
print("Хочу пиццу \"Маргариту\"")
print("Путь: C:\\Users\\Кот")
```

Выбираем со стилем



Оператор выбора match/case можно использовать его для замены if-elif-....-elif-else в случаях, когда проверяется одна переменная, но у неё может быть много значений:

Фишки:
| — логическое ИЛИ;
_ — дефолтный кейс (как else).

Пример 1

```
day = input()
match day:  # Смотрим, какое значение у day
    case "понедельник":
            # if day == "понедельник"
        print("Начало рабочей недели...")
    case "суббота" | "воскресенье":
            # if day == "суббота"
            or day =="воскресенье"
        print("Уиииии! Выходные! 🎉 ")
    case _:  # else
        print("Любой другой день.")
```

Пример 2

```
pizza_pieces = int(input())
match pizza_pieces:
    case 0:
        print("Мне не хватило(")
    case 1:
        print("Спасибо, что оставили, но мало.")
    case 2 | 3 | 4:
        print("Это по мне!")
        print("Ура!")
    case _:
        print("Съем во что бы то ни стало!")
```

Разумеется, каждый вложенный блок в case может содержать любое количество строчек кода.

Итог: наш код проберётся через самый сложный перекрёсток



Вы узнали:

- как заставить код принимать решения;
- почему True и False — основа всего;
- как комбинировать условия;
- что нужно сделать, чтобы не потеряться в многоэтажных условиях;
- как найти кота в любой строке (или не найти).

P. S. Не оставляй pass на завтра в коде, который можно написать сегодня.